

Pong Game mit ZFB

Entwickle ein voll funktionsfähiges Pong-Spiel mit der ZFB-Engine in C. Dabei lernst du: Projektstruktur mit CMake, Fenstererzeugung, Zeichnen von Formen und Texturen, Eingabeabfrage, Kollisionserkennung und grundlegende Game-Loop-Logik.

Voraussetzungen

- ZFB-Library (inkl. ZFB.a und Header-Dateien) heruntergeladen von <https://github.com/The-Sigmas/ZFB/releases>
- CMake (Version ≥ 3.10) installiert
- Ein C-Compiler (z. B. gcc oder MSVC)

Projektstruktur

Erstelle im Arbeitsverzeichnis folgende Struktur:

```
PongProject/
├── CMakeLists.txt
├── headers/    ← entpackte `headers.zip` aus ZFB
│   └── ZFB.h, ZFB_internal.h, ...
├── src/
│   └── main.c  ← hier kommt dein Pong-Code hinein
└── ZFB.a      ← Bibliothek aus dem Download
```

Aufgabe 1: CMake konfigurieren

Ergänze die Datei CMakeLists.txt im Wurzelverzeichnis:

```
# Projektname und Sprache
project(Pong C)

# Binary und Quellcode
add_executable(pong src/main.c)

# Header-Ordner einbinden
target_include_directories(pong PRIVATE headers)

# Linken der ZFB-Library und der Standard-Math-Library
target_link_libraries(pong PRIVATE ZFB.a m)

Erkläre in eigenen Worten, was jede Zeile bewirkt.
```

Aufgabe 2: Game-Loop und Fensterinitialisierung

Implementiere in *src/main.c*:

1. **WinMain** als Einstiegspunkt (Windows) oder *int main(void)* für plattformunabhängigen Ansatz.
2. Ein *ZFB_Device* für Fenstergröße (800×800) und Titel "Pong Game".
3. Aufruf von *ZFB_InitFB(&dev)* und *ZFB_CreateWindow(&dev, ...)*.
4. Game-Loop, der läuft, bis das Fenster geschlossen oder ESC gedrückt wird.

Aufgabe 3: Zeichnen von Hintergrund und Entities

- Zeichne in jeder Iteration:
 - den Hintergrund (z. B. *ZFB_DrawBG(dev, &ZFB_Black, NULL)*)
 - die beiden Paddles (*ZFB_DrawRect*) in unterschiedlichen Farben
 - den Pong-Ball als Rechteck (*ZFB_DrawRect*) oder mit Textur

Zusätzlich kannst du zu Beginn einen Startscreen (start.png) anzeigen und auf eine Taste warten.

Aufgabe 4: Eingabe und Kollision

1. Lese Tastendrucke ab (*ZFB_ProcessKeyboard()* + *ZFB_IsKeyPressed()*).
2. Bewege Player 1 mit W/S, Player 2 mit Pfeiltasten.

3. Kollision zwischen Ball und Paddles mit *ZFB_CheckCollision(Entity, Entity)* erkennen.
4. Ballrichtung invertieren (*ballDir.x = -ballDir.x*) bei Kollision.
5. Spielfeldränder abfragen und hoch-/runterspringen lassen oder Spiel beenden, wenn Ball links/rechts herausfällt.

Aufgabe 5: Physik-Update (optional)

Die ZFB-Engine unterstützt auch *ZFB_UpdatePhysics(&entity, deltaTime)*. Entferne das manuelle Position-Update und nutze die integrierte Physik mit Vektor, Masse und ggf. Gravitation für fortgeschrittene Effekte.

Vollständiger Mustercode (src/main.c)

```
#include "ZFB.h"

#define BALLW 50
#define BALLH 50
#define PADDLEW 50
#define PADDLEH 200
#define SPEED 25

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrev, LPSTR cmdLine, int showCmd)
{
    // Gerät initialisieren

    ZFB_Device dev = { .width = 800, .height = 800, .title = "Pong Game" };
    ZFB_InitFB(&dev);
    ZFB_EventInit();
    ZFB_CreateWindow(&dev, hInstance, hPrev, cmdLine, showCmd);

    // Texturen (optional)

    ZFB_Texture* startTex = ZFB_LoadTexture("start.png");

    // Entities definieren

    ZFB_Vector2 dir = { SPEED, SPEED };

    ZFB_Entity ballE = { .id = 0, .physics = { .position =
{360,360}, .mass=1, .gravity=false } , .width=BALLW, .height=BALLH };
```

```

ZFB_Entity p1E = { .id = 1, .physics = { .position = {0,
300}, .mass=1, .gravity=false } , .width=PADDLEW, .height=PADDLEH };

ZFB_Entity p2E = { .id = 2, .physics = { .position =
{750,300}, .mass=1, .gravity=false } , .width=PADDLEW, .height=PADDLEH };


// Pre-Startscreen
while(!ZFB_IsKeyPressed(32)) { // Leertaste
    ZFB_WinMessage();
    ZFB_DrawBG(dev, &ZFB_Black, startTex);
    ZFB_Present(dev);
    Sleep(33);
}

bool quit = false;
ZFB_Rect rBall, rP1, rP2;
ZFB_Event event;


// Game-Loop
while(!quit) {
    ZFB_WinMessage();
    ZFB_ProcessKeyboard();
    ZFB_PollEvent(&event);
    if(event.type==ZFB_EVENT_QUIT || ZFB_IsKeyPressed(27)) quit=true;


    // Spieler-Bewegung
    if(ZFB_IsKeyPressed('W') && p1E.physics.position.y>0)    p1E.physics.position.y -=
SPEED;

    if(ZFB_IsKeyPressed('S') && p1E.physics.position.y+PADDLEH<dev.height)
p1E.physics.position.y += SPEED;

    if(ZFB_IsKeyPressed(ZFB_Key_ArrowUp) && p2E.physics.position.y>0)
p2E.physics.position.y -= SPEED;

```

```

    if(ZFB_IsKeyPressed(ZFB_Key_ArrowDown) &&
p2E.physics.position.y+PADDLEH<dev.height) p2E.physics.position.y += SPEED;

// Ball-Logik
if(balle.physics.position.y<=0 || balle.physics.position.y+BALLH>=dev.height) dir.y = -dir.y;
if(balle.physics.position.x<=0 || balle.physics.position.x+BALLW>=dev.width)    quit=true;
if(ZFB_CheckCollision(balle,p1E) && dir.x<0) dir.x = -dir.x;
if(ZFB_CheckCollision(balle,p2E) && dir.x>0) dir.x = -dir.x;

balle.physics.position.x += dir.x;
balle.physics.position.y += dir.y;

// Synchronisieren & Zeichnen
ZFB_SyncEntity(&rBall, balle);
ZFB_SyncEntity(&rP1,  p1E);
ZFB_SyncEntity(&rP2,  p2E);
ZFB_DrawBG(dev, &ZFB_Black, NULL);
ZFB_DrawRect(dev, rBall, &ZFB_Green);
ZFB_DrawRect(dev, rP1,  &ZFB_Blue);
ZFB_DrawRect(dev, rP2,  &ZFB_Red);
ZFB_Present(dev);
Sleep(33);
}
return 0;
}

```

zusätzlich kannst du zu Beginn einen Startscreen (start.png) anzeigen und auf eine Taste warten.

Abgabe

- Kompletter PongProject/-Ordner als ZIP
- Kurze Dokumentation (max. 1 Seite) zu:
 - Erfüllte Schritte
 - Beobachtetes Verhalten
 - Mögliche Erweiterungen (z. B. Punktezählung, KI-Gegner, Grafiken)

Viel Erfolg und Spass beim Coden!